

Master of Science: Artificial Intelligence and Machine Learning

→ No... ▾

Пошук

🔍

📈

📅

🔔

Міні-курси

👤

NEW

- ✔️ Огляд ди
- ✔️ Тема 1.
- ✔️ Наукова
- ✔️ Тема 2.
- ✔️ Тема 3.
- ✔️ Тема 4.
- ✔️ Тема 5.
- ✔️ Тема 6.
- ✔️ Наукова
- ✔️ Тема 7.
- ✔️ Завдання
- ✔️ Тема 8.
- ✔️ **Тема 9.**
- ✔️ Завдання
- ✔️ Тема 10.
- ✔️ Фінальне

Супервузли (super-nodes)

Супервузли (super-nodes)

Це одна з головних пасток при проєктуванні графів. Супервузол — це вузол з аномально великою кількістю ребер: тисячі, мільйони зв'язків на один вузол.

Джерело проблеми

Уявіть граф соціальної мережі. Звичайний користувач має кілька сотень друзів. Але що якщо додати вузол (:Celebrity {name: "Britney Spears"}) з 50 мільйонами фоловерів? Будь-який запит, який зачіпає цей вузол, змушений перебирати всі його ребра:

```
MATCH (:Celebrity {name: "Popular Account"})<-[:FOLLOWS]-
(follower:User)
WHERE follower.country = "DE"
RETURN count(follower)
```

Виконання наведеного вище запиту на «супервузлі» може займати хвилини. В Neo4j ребра вузла зберігаються як зв'язний список, тому для пошуку потрібної підмножини доведеться пройти по всьому списку. Index-free adjacency, яка є головною перевагою графової моделі, в цьому випадку перетворюється на проблему.

Виявлення супервузлів

Для виявлення супервузлів достатньо визначити вузли з високим ступенем. В теорії графів ступінь вузла — це кількість ребер, інцидентних йому.

Приклад:

```
// знайти вузли з великою кількістю ребер
MATCH (n)
WITH n, size((n)--()) AS degree
WHERE degree > 10000
RETURN labels(n), n.name, degree
ORDER BY degree DESC
LIMIT 20
```

Стратегії боротьби з супервузлами

Розглянемо декілька підходів до боротьби з супервузлами.



Стратегія 1: уникати прямих зв'язків і використовувати проміжні вузли-агрегатори

Замість того щоб кожен користувач напряду стежив за знаменитістю, введемо вузли-«фан-групи» за географією або інтересами:

```
(User)-[:MEMBER_OF]->(:FanGroup {region: "DE"})-[:FANS_OF]->
(:Celebrity)
```

Тепер запит «скільки німецьких підписників» — це два кроки, але кожен крок зачіпає відносно невелику кількість ребер.

Стратегія 2: партиціонування за часом

Якщо супервузол з'являється через накопичення історичних даних — можна розбити його за часовими сегментами:

```
(Event:Event2023)-[:OCCURRED_IN]->(:Year {value: 2023})
(Event:Event2024)-[:OCCURRED_IN]->(:Year {value: 2024})
```

Стратегія 3: денормалізація критичних даних

У ситуаціях, коли вам потрібен лише агрегат (наприклад, кількість підписників популярного акаунту), ефективніше зберігати цей агрегат **як властивість вузла** і оновлювати його при змінах, замість того, щоб щоразу обходити всі пов'язані ребра. Це особливо важливо для супервузлів, у яких може бути мільйони вхідних зв'язків.

Приклад:

```
MATCH (c:Celebrity {name: "Britney Spears"})
SET c.follower_count = c.follower_count + 1
```

Пояснення:

- `c.follower_count` — властивість вузла, що зберігає поточну кількість підписників.
- Кожне додавання нового підписника оновлює агрегат **атомарно**, без необхідності проходити по всіх ребрах.
- Такий підхід знижує навантаження на пам'ять і прискорює запити до супервузлів, обхід усіх зв'язків може займати хвилини.

Недолік:

- Денормалізація порушує чистоту графової моделі, оскільки агрегат зберігається не як висновок з графа, а як дубльована інформація.
- Необхідно стежити за консистентністю: кожна зміна, що впливає на агрегат, має коректно оновлювати властивість вузла.

Для гарячих лічильників і аналітики по супервузлах це **розумний компроміс між продуктивністю і чистотою моделі**.

Стратегія 4: фільтрація через індекс перед обходом

Якщо супервузол неминучий, важливо **скоротити кількість ребер, якими потрібно проходити**, використовуючи індекси властивостей сусідніх вузлів. Це дозволяє Neo4j спочатку швидко вибрати релевантні вузли через індекс, а потім перевіряти лише необхідні зв'язки замість повного обходу всіх ребер супервузла.

Приклад:

```
// Спочатку знаходимо потрібних користувачів через індекс
MATCH (follower:User {country: "DE"})
// Потім перевіряємо їхній зв'язок із супервузлом
MATCH (follower)-[:FOLLOWS]->(c:Celebrity {name: "Popular Account"})
RETURN count(follower)
```

Пояснення:

- `follower:User {country: "DE"}` — використання індексованої властивості дозволяє одразу звузити вибірку.
- `MATCH (follower)-[:FOLLOWS]->(c:Celebrity {name: "Popular Account"})` — перевіряємо лише тих користувачів, що підходять за фільтром, що значно знижує навантаження на супервузол.
- Такий підхід особливо корисний, коли супервузол має мільйони вхідних зв'язків.

Фільтрація через індекс — ефективна стратегія для роботи з супервузлами: замість того щоб покладатися на обхід за посиланнями, вона дозволяє одразу звузити вибірку і перевіряти лише релевантні ребра.

Коли супервузол неминучий

Іноді жодне проєктування не рятує. Вузол «*місто Берлін*» у географічному графі матиме мільйони зв'язків з людьми, місцями, подіями — і це фундаментальна властивість даних, а не помилка проєктування. В таких випадках можна розглянути два варіанти:

◆ **Шардування.** Розбити граф на кілька баз, де кожна містить підграф за географією або іншим розподільувачем, а запити, що перетинають шарди, виконуються на рівні застосунку.

◆ Можливо, Neo4J не найкращий інструмент для вашого завдання, і є сенс розглянути щось інше. Наприклад, FalkorDB з його матричним представленням краще справляється з висококомпонентними вузлами, ніж Neo4j зі зв'язним списком ребер.

Рекомендації

Хороша графова схема — це схема, де максимальний ступінь вузла (кількість ребер) залишається керованим. Якщо при проєктуванні ви бачите, що якийсь вузол притягуватиме мільйони зв'язків — це сигнал переглянути модель.

До наступного блоку

